¿Cuándo decide un cliente abandonar su banco?

Predicción de Bajas de Clientes · Machine Learning · XGBoost

🏦 **EL PROBLEMA**

1 de cada 5 clientes abandonará el banco este año

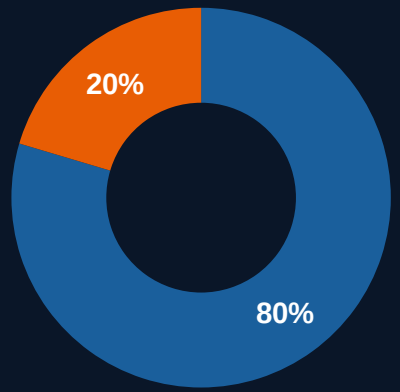
💰 **EL COSTE**

Retener cuesta 5-7× menos que captar un cliente nuevo

🎯 **NUESTRA SOLUCIÓN**

Predecir ANTES de que el cliente tome la decisión

DISTRIBUCIÓN DEL TARGET



Permanece (79.6%) Abandona (20.4%)

10.000	clientes analizados
3	mercados: Francia · Alemania · España
>0.86	ROC-AUC final en test

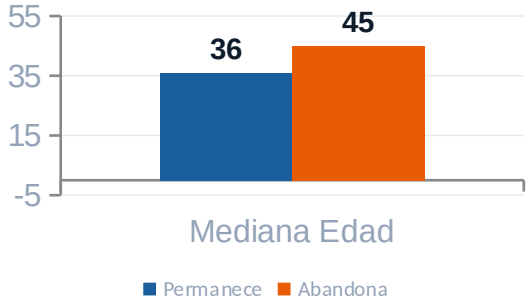
¿Quién abandona el banco y por qué?

EDA · 3 hallazgos clave con datos reales

01

1

👴 La Edad Importa

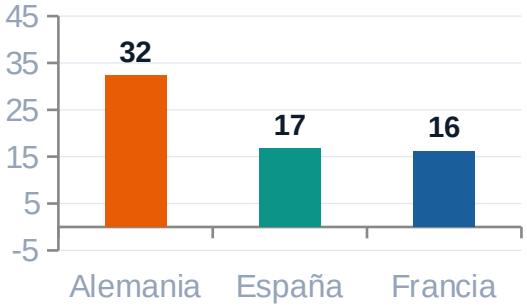


Mediana de edad: 36 años (permanece) vs 45 años (abandona). Diferencia de 9 años.

💼 *Los clientes maduros tienen más experiencia financiera y más opciones para comparar.*

2

🌍 Alemania en Alerta

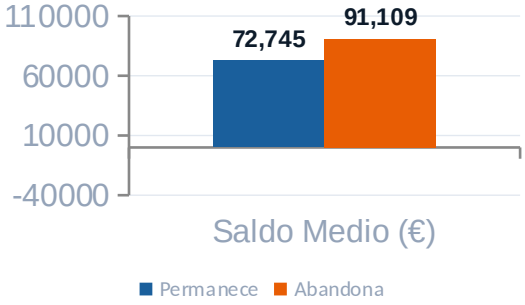


Tasa churn: Alemania 32.4% · España 16.7% · Francia 16.2%. Alemania dobla.

💼 *El mercado alemán presenta mayor competencia bancaria — acción prioritaria.*

3

💸 El Paradox del Saldo



Saldo medio quien abandona: 91.109€ vs 72.745€ de quien permanece.

💼 *Más dinero = más atractivo para la competencia → feature balance_per_product*

Preprocesado y Feature Engineering

Pipeline sin data leakage · 4 features nuevas derivadas del EDA

02

⚙️ EL PIPELINE — 7 PASOS EN ORDEN

1 Split Train/Test — PRIMERO
80/20 stratify · regla de oro · evita data leakage

2 Duplicados
Sin duplicados — dataset limpio ✅

3 Missings — Imputación mediana
Robusta a outliers · fit solo en train · listo para producción

4 Outliers — Capping P1-P99
Conserva todos los registros · límites aprendidos en train

5 Feature Engineering
4 features nuevas antes de escalar — valores originales

6 One-Hot Encoding
Geography, Gender — sin orden implícito entre categorías

7 StandardScaler — AL FINAL
fit solo en train · transform en ambos

✨ 4 FEATURES NUEVAS — 1 DEL EDA · 3 EN PREPROCESADO

`balance_per_product` ⭐ EDA

= Balance ÷ (NumProductos + 1)

Sugerida en el EDA · Concentración de dinero por producto → mayor riesgo

`HasBalance`

= Balance > 0 → 1 / 0

Creada en preprocesado · 36% tiene saldo 0 — señal de desenganche

`EngagedCustomer`

= Activo AND NumProductos > 1

Creada en preprocesado · Perfil más comprometido con el banco

`SalaryAgeRatio`

= Salario ÷ (Edad + 1)

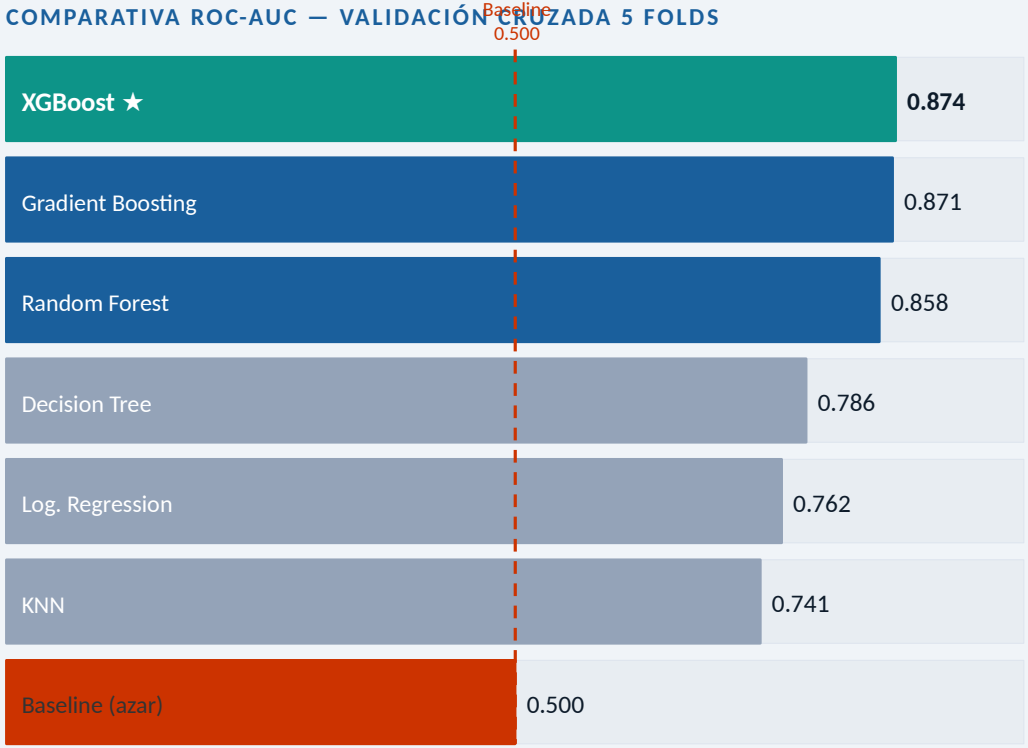
Creada en preprocesado · Riqueza relativa según etapa de vida

Modelado — ¿Qué algoritmo predice mejor?

Baseline → 6 modelos con CV-5 → Optimización · 150 entrenamientos

03

COMPARATIVA ROC-AUC — VALIDACIÓN CRUZADA 5 FOLDS



¿POR QUÉ XGBOOST?

🔒 **Regularización nativa L1+L2**

Gradient Boosting no la tiene → XGBoost sobreajusta menos

⚖️ **scale_pos_weight nativo**

Maneja el desbalanceo 80/20 directamente — sin trucos externos

🚀 **Más hiperparámetros útiles**

Más palancas de optimización → mayor mejora tras el tuning

⚡ **OPTIMIZACIÓN — RandomizedSearchCV**

→ 30 combinaciones × 5 folds = 150 entrenamientos

→ 7 hiperparámetros: n_estimators, max_depth, learning_rate...

→ Métrica de optimización: ROC-AUC

Resultados y Conclusiones

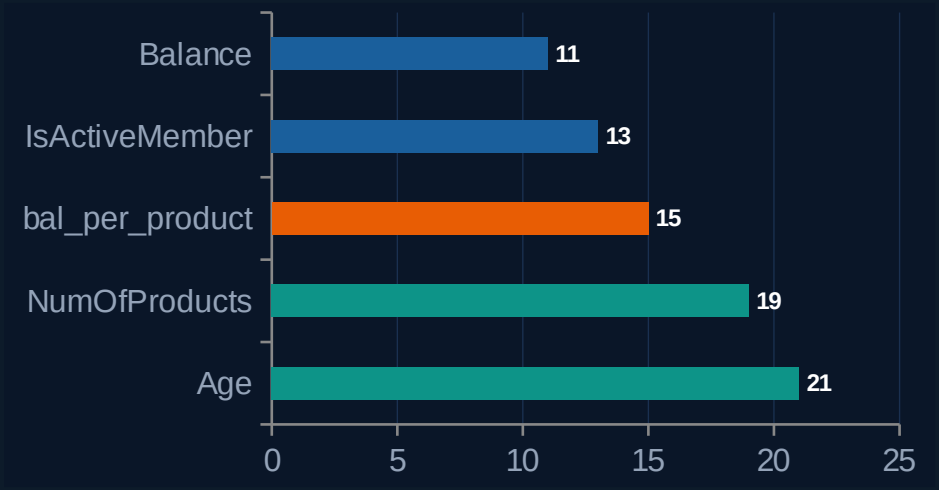
XGBoost optimizado · evaluación final sobre datos nunca vistos · impacto de negocio

>0.86
ROC-AUC en Test
vs 0.50 del baseline

~0.62
F1-Score clase 1
Equilibrio detección/alarmas

<0.03
Diferencia CV–Test
Sin sobreajuste ✅

TOP VARIABLES PREDICTIVAS



IMPACTO DE NEGOCIO

De cada 100 clientes que van a irse...

el modelo detecta a **+60** antes de que se vayan

- ✅ Modelo estable — AUC similar en CV y test (sin sobreajuste)
- 🔑 Age, NumOfProducts y balance_per_product lideran la predicción
- 💼 Retener cuesta 5-7× menos — cada churner detectado tiene valor real
- ⚠️ Mejora posible: ajustar threshold para reducir falsos negativos